

Java

Multithreading

Theory

1. Multithreading in Java

Definition: Multithreading is the process of executing multiple threads concurrently. **Key points:**

- A **thread** is a lightweight sub-process.
- Threads share the same memory space but run independently.
- Thread creation:
 - Extend Thread class.
 - Implement Runnable interface.

Syntax:

java

```
class MyThread extends Thread {  
    public void run() { System.out.println("Thread running"); }  
}  
  
class MyRunnable implements Runnable {  
    public void run() { System.out.println("Runnable running"); }  
}
```

2. Thread Lifecycle

- **New** → thread created but not started.
- **Runnable** → ready to run, waiting for CPU.
- **Running** → executing.
- **Non-runnable (waiting/sleeping)** → temporarily inactive.
- **Terminated** → finished execution.

3. Thread Methods

- start() → begins execution.
- run() → contains thread logic.
- sleep(ms) → pauses thread.
- join() → waits for thread to finish.
- isAlive() → checks if thread is running.
- setPriority() / getPriority() → manage thread priority.
- setName() / getName() → manage thread names.

4. Synchronization

Definition: Synchronization ensures only one thread accesses a shared resource at a time. **Use case:** Prevents race conditions.

Syntax:

java

```
class Counter {
    int count = 0;
    synchronized void increment() { count++; }
}
```

Coding Round Questions

- Thread creation → Demonstrate thread creation using both Thread and Runnable.
- Thread lifecycle → Explain thread lifecycle stages with examples.
- Thread methods → Show usage of start, sleep, join, and isAlive.
- Thread priority → Demonstrate setting and getting thread priority.
- Synchronization → Write a program to increment a counter using two threads with synchronization.

Machine Coding Round Challenge

Problem: Ticket Booking System

Requirements:

- Create a TicketCounter class with available tickets.
- Method bookTicket() should reduce ticket count.
- Use **synchronization** to prevent overselling when multiple users book simultaneously.
- Create multiple threads representing users booking tickets.
- Print remaining tickets after all threads finish.

```
class MyThread extends Thread{

    public void run(){
        System.out.println("Run method from MyThread class");
    }
}

class MyRunnable implements Runnable{

    public void run(){
        System.out.println("Run method from MyRunnable class");
    }
}

public class Multithreading {

    public static void main(String[] args) throws InterruptedException {

        System.out.println("Multithreading");

        MyThread obj = new MyThread();
        obj.setPriority(7);
        System.out.println("MyThread class priority :"+ obj.getPriority());
        obj.setName("Mythread");
        System.out.println("Mythread class thread name :"+obj.getName());

        Thread.sleep(2000);

        Thread runnable = new Thread(new MyRunnable());
        System.out.println("Current thread:"+Thread.currentThread());

        runnable.start();
        System.out.println(runnable.isAlive());
    }
}
```

```

        runnable.setPriority(Thread.MAX_PRIORITY);
        System.out.println("MyRunnable class priority : " +
runnable.getPriority());
        runnable.setName("MyRunnable");
        System.out.println("MyRunnable class thread name
:" + runnable.getName());

obj.start();

System.out.println(obj.isAlive());

// main thread waits until this thread executes
obj.join();
runnable.join();

System.out.println("End of the program ");

// thread vs process
    }
}

class Counter{

    int count =0;

    synchronized void increment(){
        count++;
    }
}

public class MultiThreadSync {

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Multithreading");

        Counter obj = new Counter();

```

```
Thread t1 = new Thread( ()->{
    for(int i=1; i<=30000; i++){
        obj.increment();
    }
} );

Thread t2 = new Thread( ()->{
    for(int i=1; i<=30000; i++){
        obj.increment();
    }
} );

t1.start();
t2.start();

t1.join();
t2.join();

System.out.println(obj.count);

}

}
```